

ระบบแจ้งซ่อมและขอความช่วยเหลือด้านไอที

IT Service Desk & Incident Ticket API

ตามมาตรฐานสถาปัตยกรรม Application (NestJS + Prisma ORM + PostgreSQL + pgAdmin)

รหัสโครงการ / รายวิชา

project-cybersec-69 (Sec 69)

จำนวนสมาชิกทีม

3 คน (DevOps/DB, App/Auth, REST/Testing)

รูปแบบสถาปัตยกรรม

Application Clean Architecture & REST Client

เครื่องมือทดสอบหลัก

api.http / api.http.simple (VS Code REST Client)

สรุปภาพรวมและวัตถุประสงค์โครงการ

โครงการนี้เป็นการพัฒนาระบบ IT Service Desk & Incident Ticket API เพื่อเป็นศูนย์กลางในการรับเรื่อง แจ้งปัญหาทางเทคนิค (Hardware, Software, Network, Security Incident) และขอสิทธิ์ใช้งานภายในองค์กร โดยจำลองโครงสร้างและขั้นตอนการทำงานตามโปรเจกต์ 69-s1-cybersec เดิม พร้อมยกระดับไปสู่สถาปัตยกรรมสมัยใหม่ที่สร้างด้วยแนวคิดของ Application (NestJS, Prisma ORM, Dual-mode Database, และ Web Dashboard ครบวงจร)

★ ผลสัมฤทธิ์ที่ส่งมอบในโครงการ

- ระบบ Docker Compose รัน PostgreSQL 16 + pgAdmin 4 + NestJS Backend Service
- ฐานข้อมูล Prisma ORM รองรับ 4 Entities (User, Category, Ticket, Comment) พร้อม Seed Data
- ระบบยืนยันตัวตน JWT Passport พร้อมการแยกสิทธิ์ 3 บทบาท (ADMIN, IT_SUPPORT, EMPLOYEE)
- ชุดทดสอบครบวงจรผ่านไฟล์ api.http.simple ในสไตล์เดียวกับโปรเจกต์เดิม
- หน้าเว็บควบคุม Web Dashboard, Swagger OpenAPI Docs และ Prisma Studio Web GUI

1. ที่มา จุดเด่น และประโยชน์ของสถาปัตยกรรม Amplication

Amplication เป็นแพลตฟอร์ม Open-source ขั้นนำสำหรับสร้าง Backend Services และ APIs (Node.js / NestJS) แบบอัตโนมัติ โดยผสมผสานข้อดีของ Low-Code เข้ากับ Professional Code Generation (ได้โค้ดภาษา TypeScript แท้ 100%)

1.1 จุดเด่นสำคัญ (Key Highlights)

- Clean Architecture ระดับ Enterprise:** ผลิตโค้ด NestJS ที่แยกเป็น Modules, Controllers, Services, DTOs และ Prisma Service อย่างเป็นระเบียบ อ่านเข้าใจง่าย และปรับแต่งต่อยอดได้อย่างอิสระ
- Type-Safety สูงสุดด้วย Prisma ORM:** ใช้ Prisma ในการเชื่อมโยงฐานข้อมูล ทำให้การสืบค้นข้อมูลมี Type-checking ป้องกันข้อผิดพลาดตั้งแต่ตอนเขียนโค้ด
- ระบบ Authentication & Role-Based Access Control (RBAC):** ติดตั้ง Passport-JWT และ Bcrypt Hashing มาตรฐาน พร้อมโครงสร้าง Guard สำหรับจำแนกสิทธิ์ผู้ใช้งาน
- ไร้ปัญหา Vendor Lock-in:** เป็นเจ้าของซอร์สโค้ด 100% สามารถนำไปรันบนเซิร์ฟเวอร์หรือ Cloud ใดก็ได้โดยไม่ต้องพึ่งพา Amplication ในอนาคต

1.2 เปรียบเทียบกับ Strapi 5 (โปรเจกต์ 69-s1-cybersec เดิม)

คุณสมบัติ	Strapi 5 (โปรเจกต์เดิม)	Amplication (โปรเจกต์ใหม่)
รูปแบบระบบ	Headless CMS สำเร็จรูป รันจาก Image	Backend Code Generator (NestJS + TypeScript บริสุทธิ์)
การปรับแต่งโค้ด	ต้องทำ Monkey Patch ผ่าน Docker Volume Mounts	แก้ไข Source Code โดยตรง ใน Controller, Service, DTO
ฐานข้อมูลและ ORM	Knex.js / Custom Query Engine	Prisma ORM (ทันสมัย Type-Safe และจัดการ Migration ได้ง่าย)
Admin Dashboard	มี Built-in Admin Panel ที่พอร์ต 1337	มี Web Service Desk Dashboard + Prisma Studio (พอร์ต 5555)
แนวทางการทดสอบ	ใช้ api.http.simple ใน VS Code	ใช้ api.http.simple รูปแบบเดียวกันเป๊ะ + Swagger UI

2. สถาปัตยกรรมระบบและโครงสร้าง Docker Environment

```
+-----+
|                                     |
|               | DOCKER COMPOSE |
| [ Service: db ] [ Service: pgadmin ] [ Service: app ] |
| postgres:16-alpine dpage/pgadmin4 NestJS Application |
|   | Port: 5432 Port: 5050 Port: 9091 | | | |
|   | Vol: ./data/db Vol: ./data/pgadmin Vol: ./src |
|   | | | | |
|   | +<-----+ |
| +<-----+ |
|               | (Prisma ORM Client) |
+-----+
```

การจัดสรรพอร์ต (Port Allocation Table)

Service Name	Container Image	External Port	Internal Port	คำอธิบายหน้าที่
db	postgres:16-alpine	5432	5432	ฐานข้อมูลหลัก PostgreSQL เก็บข้อมูล Users, Tickets, Categories
pgadmin	dpage/pgadmin4	5050	80	Web GUI จัดการและดูโครงสร้างตารางฐานข้อมูล
app	Custom (Node.js 20)	9091	9091	NestJS API Service (Swagger ที่ /api, Web UI ที่ /admin)

```
# docker-compose.yaml ตัวอย่างคอนฟิกชัน
services:
  db:
    image: postgres:16-alpine
    container_name: it-servicedesk-db
    ports: ["${POSTGRES_PORT}:5432"]
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes: ["/data/db:/var/lib/postgresql/data"]
  pgadmin:
    image: dpage/pgadmin4
    container_name: it-servicedesk-admin
    ports: ["${PGADMIN_PORT}:80"]
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
    volumes: ["/data/pgadmin:/var/lib/pgadmin"]
    depends_on: [db]
  app:
    build: .
    container_name: it-servicedesk-app
    ports: ["${APP_PORT}:9091"]
    environment:
      DATABASE_URL: ${DATABASE_URL}
      JWT_SECRET: ${JWT_SECRET}
      APP_PORT: ${APP_PORT}
    depends_on: [db]
```

3. การออกแบบฐานข้อมูล (Database Schema & Seed Data)

ฐานข้อมูลถูกออกแบบให้ครอบคลุมการทำงานของ IT Service Desk ทั้งหมด 4 ตารางหลัก เชื่อมโยงกันผ่าน Relational Constraints ใน Prisma Schema:

ตาราง (Entity)	ฟิลด์สำคัญ (Columns)	ประเภทข้อมูล & Constraints	คำอธิบายและหน้าที่
User	id, email, password firstName, lastName role, department	String PK, String Unique String (Bcrypt Hashed) Enum (ADMIN, IT_SUPPORT, EMPLOYEE)	ผู้ใช้งานระบบทั้งหมด จำแนกสิทธิ์ตาม Role เพื่อควบคุมการเข้าถึงตัวและฟังก์ชันต่างๆ
Category	id, name, description	String PK, String Unique String Optional	หมวดหมู่ปัญหา เช่น Hardware, Software, Network, Security Incident, Account Access
Ticket	id, title, description priority, status createdById, assignedToId categoryId	String PK, Text Enum (LOW, MEDIUM, HIGH, CRITICAL) Enum (OPEN, IN_PROGRESS, RESOLVED, CLOSED) FK -> User, FK -> Category	รายการตัวแจ้งซ่อม/ปัญหา มี Workflow การเปลี่ยนสถานะตามขั้นตอนการทำงานจริง
Comment	id, content, isInternal ticketId, authorId	String PK, Text Boolean (Internal IT note or Public) FK -> Ticket, FK -> User	ความคิดเห็น บันทึกการติดตามงาน และการตอบกลับระหว่างผู้แจ้งกับทีมไอที

ข้อมูลตั้งต้นสำหรับทดสอบ (Pre-populated Seed Data)

- Super Admin** : admin@servicedesk.local (รหัสผ่าน: Password123!) - ดูแลระบบทั้งหมด
- IT Support** : support@servicedesk.local (รหัสผ่าน: Password123!) - เจ้าหน้าที่รับเรื่องซ่อม
- Employee 1 & 2** : employee1@servicedesk.local, employee2@servicedesk.local (รหัสผ่าน: Password123!)
- Categories 5 รายการ**: Hardware, Software, Network, Security Incident, System Access
- Mock Tickets 3 รายการ**: ครอบคลุมสถานะ OPEN, IN_PROGRESS และ RESOLVED

4. การแบ่งบทบาทหน้าที่ 3 คน และ Git Branch Strategy

สมาชิก	บทบาท / ตำแหน่ง	Branch ที่รับผิดชอบ	ชิ้นงานที่ต้องส่งมอบ (Deliverables)
สมาชิกคนที่ 1	Database & Admin Lead (ดูแลโครงสร้างพื้นฐาน)	feat/db feat/admin	- docker-compose.yaml (ส่วน db และ pgadmin) .env.simple และ .env - prisma/schema.prisma (4 Models) - prisma/seed.ts (Mock users & tickets)
สมาชิกคนที่ 2	App Core & Auth Lead (ดูแลตัวแอปและระบบล็อกอิน)	feat/app	- โครงสร้าง NestJS Application Core - Dockerfile และ Service app ใน compose - Authentication Module (JWT + Passport) - Endpoints: Admin/User Login, Register, /me - Swagger OpenAPI Module ที่ /api
สมาชิกคนที่ 3	REST API & Testing Lead (ดูแล CRUD & ไฟล์ทดสอบ)	feat/rest	- CRUD Controllers สำหรับ Tickets, Categories, Comments - DTOs พร้อม Data Validation - ไฟล์ api.http และ api.http.simple สำหรับยิงทดสอบครบทุกข้อ - ตรวจสอบความถูกต้องของ Response ร่วมกับทีม

ลำดับขั้นตอนการทำงานร่วมกัน (Team Handover Timeline)

- ระยะที่ 1 (คนที่ 1 เริ่มต้น):** แยก Branch feat/db → เขียน Docker Postgres + Prisma Schema → รัน Seed ข้อมูล → แยก Branch feat/admin → เพิ่ม pgAdmin ใน Docker Compose (พอร์ต 5050) → Merge เข้าสู่ main
- ระยะที่ 2 (คนที่ 2 รับไม้ต่อ):** แยก Branch feat/app → สร้าง NestJS App Core เชื่อมต่อกับ DB ของคนที่ 1 → พัฒนาโมดูล Authentication (JWT Strategy, Login, Register, Profile) → Merge เข้าสู่ main
- ระยะที่ 3 (คนที่ 3 ดำเนินการขั้นสุดท้าย):** แยก Branch feat/rest → พัฒนา CRUD Controllers สำหรับ Tickets, Categories, Comments → จัดทำไฟล์ api.http.simple ที่มีตัวแปร `{{dotenv ...}}` ดึง Token และ ID อัตโนมัติ → ทดสอบยิง Request ครบทุกข้อ → Merge เข้าสู่ main

5. การออกแบบ API และโครงสร้างไฟล์ api.http.simple

เพื่อให้การทดสอบตรงกับรูปแบบเดิมใน 69-s1-cybersec/api.http.simple โครงสร้างไฟล์ทดสอบจะถูกแบ่งเป็น 5 หมวดหมู่:

หมวดหมู่	หัวข้อย่อย	HTTP Method & Endpoint	สิทธิ์ที่ต้องใช้
1. Admin	1.1 Admin Login	POST /admin/login	ไม่ต้องมี Token (รับ Token กลับมา)
	1.2 Admin Profile	GET /admin/users/me	Bearer Token (Admin)
2. User / Staff	2.1 User Register	POST /api/auth/register	ไม่ต้องมี Token
	2.2 User Login	POST /api/auth/login	ไม่ต้องมี Token (รับ Token กลับมา)
	2.3 User Profile	GET /api/users/me	Bearer Token (User)
3. Categories	3.1 List Categories	GET /api/categories	Bearer Token
	3.2 Create Category	POST /api/categories	Bearer Token (Admin Only)
4. Tickets (CRUD)	4.1 Create Ticket	POST /api/tickets	Bearer Token (Employee สร้างคำขอ)
	4.2 List Tickets	GET /api/tickets	Bearer Token (ดึงตามสิทธิ์ผู้เรียก)
	4.3 Assign Ticket	PATCH /api/tickets/:id/assign	Bearer Token (IT Support รับเคส)
	4.4 Update Status	PATCH /api/tickets/:id/status	Bearer Token (เปลี่ยนเป็น RESOLVED)
5. Comments	5.1 Add Comment	POST /api/tickets/:id/comments	Bearer Token
	5.2 View Comments	GET /api/tickets/:id/comments	Bearer Token

```
### 1.1 Admin Login
# @name adminLogin
POST http://localhost:{{dotenv APP_PORT}}/admin/login
Content-Type: application/json

{ "email": "{{dotenv ADMIN_EMAIL}}", "password": "{{dotenv ADMIN_PASSWORD}}" }

### 1.2 Admin Profile
GET http://localhost:{{dotenv APP_PORT}}/admin/users/me
Authorization: Bearer {{adminLogin.response.body.access_token}}

### 4.1 Create Ticket (Employee)
# @name createTicket
POST http://localhost:{{dotenv APP_PORT}}/api/tickets
Content-Type: application/json
Authorization: Bearer {{dotenv USER_TOKEN}}

{
  "title": "จอคอมพิวเตอร์เปิดไม่ติด มีไฟกระพริบสีส้ม",
  "description": "ใช้งานอยู่ดีฯ จอดับไป แผนกการเงิน ชั้น 3",
  "priority": "HIGH",
  "categoryId": "{{dotenv CATEGORY_HARDWARE_ID}}"
}
```

6. ระบบ Dashboard และหน้าจอควบคุม (User Interfaces)

Dashboard Name	URL / คำสั่งเรียกใช้	กลุ่มผู้ใช้งาน	ความสามารถเด่น
1. Web Service Desk	http://localhost:9091/admin	พนักงาน, ช่างไอที, ผู้บริหาร	ดูสถิติตัว, ตารางแจ้งซ่อม, กดรับงาน/เปลี่ยนสถานะแบบ Visual
2. Prisma Studio	npx prisma studio (5555)	นักพัฒนา, DBA	ดูและแก้ไขตารางข้อมูลโดยตรง เหมือน Strapi Content Manager
3. Swagger OpenAPI	http://localhost:9091/api	นักพัฒนา, QA Tester	ดูเอกสาร API ครบทุก Endpoint และกดทดสอบ Try it out
4. pgAdmin 4	http://localhost:5050	System Admin, DBA	จัดการ PostgreSQL, ดู Index, ตรวจสอบ Performance และ Query

7. ขั้นตอนการตรวจรับงาน (Verification Checklist)

ข้อ	รายการที่ต้องตรวจสอบ	วิธีทดสอบ	เกณฑ์ผ่าน (Criteria)
1	Service ใน Docker ติดตั้งและรันสำเร็จ	รันคำสั่ง docker compose ps	สถานะเป็น Up ครบทั้ง 3 Services
2	Prisma Migration & Seed สมบูรณ์	รัน npx prisma db seed	มีบัญชี Admin, IT, User และ Category
3	ทดสอบ Admin Login & Profile	กด Send Request ข้อ 1.1 ใน api.http	ได้ HTTP 200/201 พร้อม JWT Token
4	ทดสอบสร้างตั๋วแจ้งซ่อม (Create Ticket)	กด Send Request ข้อ 4.1 ใน api.http	ตั๋วถูกบันทึก สถานะเริ่มต้นเป็น OPEN
5	ทดสอบเปลี่ยนสถานะตั๋ว (Update Status)	กด Send Request ข้อ 4.4 ใน api.http	สถานะตั๋วเปลี่ยนเป็น RESOLVED สำเร็จ
6	ตรวจสอบหน้า Web Dashboard	เปิดเบราว์เซอร์ไปที่พอร์ต 9091	แสดงตัวเลขสถิติและตารางตั๋วถูกต้อง